

Reviewer Pack — Minimal Rank of Noncommutative Yang-Baxter Equations vs Circ...

Entry 6d90de4918c3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 09:23:20 UTC

Minimal Rank of Noncommutative Yang-Baxter Equations vs Circuit Depth for AND-OR Trees

Entry ID: 6d90de4918c3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 09:23:20 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Yang-Baxter Equations) **Field B** (complexity object): Complexity Theory (AND-OR Tree Depth)

Statement:

{‘quantitative_relation’: ‘ $E[\rho(f_n)] = \Theta(\log n)$ ’, ‘falsifier’: ‘There exists an instance f_n with $\rho(f_n) = o(\log n)$ ’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘Yang-Baxter equations provide a noncommutative algebraic framework that could potentially capture the essence of AND-OR tree depth, offering a new perspective on complexity lower bounds.’, ‘additional_notes’: ‘The minimal rank $\rho(f_n)$ of a Yang-Baxter equation encoding an AND-OR tree would be expected to scale logarithmically with the tree size if the noncommutative algebraic structure preserves essential complexity properties.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3382d0ff4c10a8a1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \leq 40$, the average minimal rank $\rho(f_n)$ across 30 seeds is within 3 standard deviations of $\log(n)$, and no seed produces a $\rho(f_n) = o(\log n)$. The conjecture is falsified if any seed produces a $\rho(f_n)$ significantly less than $\log(n)$ or if the average $\rho(f_n)$ differs from $\log(n)$ by more than $\bar{3}$ standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_and_or_tree(depth):
        if depth == 0:
            return random.choice([0, 1])
        else:
            left = generate_and_or_tree(depth - 1)
            right = generate_and_or_tree(depth - 1)
            return (left, right) if random.randint(0, 1) == 0 else (right, left)

    def construct_yang_baxter_equation(tree):
        if isinstance(tree, int):
            return [[Fraction(1), Fraction(0)], [Fraction(0), Fraction(1)]]
        else:
            left_eq = construct_yang_baxter_equation(tree[0])
            right_eq = construct_yang_baxter_equation(tree[1])
            # Simplified braid relation for demonstration
            return [
                [left_eq[0][0] * right_eq[0][0], left_eq[0][0] * right_eq[0][1]],
                [left_eq[1][0] * right_eq[1][0], left_eq[1][0] * right_eq[1][1]]
            ]

    def calculate_minimal_rank(eq):
        # Simplified rank calculation for demonstration
        return max(abs(eq[0][0]), abs(eq[0][1]), abs(eq[1][0]), abs(eq[1][1]))

    results = []
    for n in range(5, 41):
        tree = generate_and_or_tree(n)
        eq = construct_yang_baxter_equation(tree)
        rank = calculate_minimal_rank(eq)
        results.append((n, rank))
```

```

mean_rank = sum(rank for _, rank in results) / len(results)
std_dev = math.sqrt(sum((rank - mean_rank) ** 2 for _, rank in results) / len(results))
conjecture_holds = all(mean_rank - 3 * std_dev <= rank <= mean_rank + 3 * std_dev for _, rank

return {
    "metric_name": "minimal_rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean_rank={mean_rank}, std_dev={std_dev}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mean_rank did not meet the conjectured bound\"")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time constraints.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13057
2	propose	ollama_remote	glm4:latest	0	0	10172
3	preregistration	ollama_remote	glm4:latest	0	0	6629
4	novelty	ollama_remote	glm4:latest	0	0	4886
5	novelty	ollama_remote	glm4:latest	0	0	5330
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11990
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27781
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9585
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11673
10	judge	ollama_remote	glm4:latest	0	0	45212

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146313 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/6d90de4918c3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6d90de4918c3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6d90de4918c3.tar.gz` (if generated)